

```

import numpy as np # Import numpy for numerical operations

# Install gensim if not already installed
!pip install gensim

# Import gensim to load pre-trained Word2Vec model
import gensim

# Import KeyedVectors to load binary embedding file (though not directly used in the final version of this cell)
from gensim.models import KeyedVectors

# Import matplotlib for visualization
import matplotlib.pyplot as plt

# Import PCA for dimensionality reduction
from sklearn.decomposition import PCA

# Import cosine similarity function from numpy.linalg
from numpy.linalg import norm

```

```

Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)

```

```

"""
Load GoogleNews Word2Vec Pre-trained Model
"""

# Path to downloaded model (commented out as gensim.downloader is used)
# model_path = "GoogleNews-vectors-negative300.bin.gz"

# Download the model if it doesn't exist or is incomplete (commented out as gensim.downloader is used)
# This specific model is very large (~3.6 GB) and can take a while to download.
# The EOFError suggests the file might be incomplete, so re-downloading is a good first step.
# import os
# if not os.path.exists(model_path) or os.path.getsize(model_path) < 3600000000: # Approximate expected size in bytes
#     print(f"Downloading {model_path}...")
#     !wget -c "https://s3.amazonaws.com/dl4j-distribution/GoogleNews-vectors-negative300.bin.gz"
#     print(f"Finished downloading {model_path}.")
# else:
#     print(f"{model_path} already exists and seems complete.")

# Use gensim.downloader to load the pre-trained GoogleNews-vectors-negative300 model
import gensim.downloader as api
print("Downloading GoogleNews-vectors-negative300 model using gensim.downloader...")
model = api.load("word2vec-google-news-300") # Load the Word2Vec model
print("Model loaded successfully.")

# Load binary model - This line is now redundant as api.load handles it. (commented out)
# model = KeyedVectors.load_word2vec_format(model_path, binary=True)

# Print vocabulary size of the loaded model
print("Vocabulary Size:", len(model.key_to_index))

# Display example vector for the word 'king'
word = "king"
print("\nVector for word 'king':\n", model[word]) # Access the vector for 'king'
print("\nVector Dimension:", len(model[word])) # Print the dimension of the vector

```

```
-6.49414062e-02 5.07812500e-02 1.35742188e-01 -7.47070312e-02
-1.64062500e-01 1.15356445e-02 4.45312500e-01 -2.15820312e-01
-1.11328125e-01 -1.92382812e-01 1.70898438e-01 -1.25000000e-01
2.65502930e-03 1.92382812e-01 -1.74804688e-01 1.39648438e-01
2.92968750e-01 1.13281250e-01 5.95703125e-02 -6.39648438e-02
9.96093750e-02 -2.72216797e-02 1.96533203e-02 4.27246094e-02
-2.46093750e-01 6.39648438e-02 -2.25585938e-01 -1.68945312e-01
2.89916992e-03 8.20312500e-02 3.41796875e-01 4.32128906e-02
1.32812500e-01 1.42578125e-01 7.61718750e-02 5.98144531e-02
-1.19140625e-01 2.74658203e-03 -6.29882812e-02 -2.72216797e-02
-4.82177734e-03 -8.20312500e-02 -2.49023438e-02 -4.00390625e-01
-1.06933594e-01 4.24804688e-02 7.76367188e-02 -1.16699219e-01
7.37304688e-02 -9.22851562e-02 1.07910156e-01 1.58203125e-01
4.24804688e-02 1.26953125e-01 3.61328125e-02 2.67578125e-01
-1.01074219e-01 -3.02734375e-01 -5.76171875e-02 5.05371094e-02
5.26428223e-04 -2.07031250e-01 -1.38671875e-01 -8.97216797e-03
-2.78320312e-02 -1.41601562e-01 2.07031250e-01 -1.58203125e-01
1.27929688e-01 1.49414062e-01 -2.24609375e-02 -8.44726562e-02
1.22558594e-01 2.15820312e-01 -2.13867188e-01 -3.12500000e-01
-3.73046875e-01 4.08935547e-03 1.07421875e-01 1.06933594e-01
7.32421875e-02 8.97216797e-03 -3.88183594e-02 -1.29882812e-01
1.49414062e-01 -2.14843750e-01 -1.83868408e-03 9.91210938e-02
1.57226562e-01 -1.14257812e-01 -2.05078125e-01 9.91210938e-02
3.69140625e-01 -1.97265625e-01 3.54003906e-02 1.09375000e-01
1.31835938e-01 1.66992188e-01 2.35351562e-01 1.04980469e-01
-4.96093750e-01 -1.64062500e-01 -1.56250000e-01 -5.22460938e-02
1.03027344e-01 2.43164062e-01 -1.88476562e-01 5.07812500e-02
-9.37500000e-02 -6.68945312e-02 2.27050781e-02 7.61718750e-02
2.89062500e-01 3.10546875e-01 -5.37109375e-02 2.28515625e-01
2.51464844e-02 6.78710938e-02 -1.21093750e-01 -2.15820312e-01
-2.73437500e-01 -3.07617188e-02 -3.37890625e-01 1.53320312e-01
2.33398438e-01 -2.08007812e-01 3.73046875e-01 8.20312500e-02
2.51953125e-01 -7.61718750e-02 -4.66308594e-02 -2.23388672e-02
2.99072266e-02 -5.93261719e-02 -4.66918945e-03 -2.44140625e-01
-2.09960938e-01 -2.87109375e-01 -4.54101562e-02 -1.77734375e-01
-2.79296875e-01 -8.59375000e-02 9.13085938e-02 2.51953125e-01]
```

Vector Dimension: 300

```
"""
Compute cosine similarity between word pairs
"""

def cosine_similarity(vec1, vec2):
    """
    Compute cosine similarity between two vectors.
    Returns value between -1 and 1.
    """
    return np.dot(vec1, vec2) / (norm(vec1) * norm(vec2)) # Calculate and return cosine similarity

# Define a list of word pairs for similarity comparison
word_pairs = [
    ("doctor", "nurse"),
    ("cat", "dog"),
    ("car", "bus"),
    ("king", "queen"),
    ("man", "woman"),
    ("apple", "orange"),
    ("teacher", "student"),
    ("paris", "france"),
    ("india", "china"),
    ("computer", "laptop")
]

print("Word Similarity Scores:\n")

for w1, w2 in word_pairs:
    similarity = model.similarity(w1, w2) # Compute similarity using the pre-trained model
    print(f"{w1} - {w2} : {similarity:.4f}") # Print the word pair and its similarity score
```

Word Similarity Scores:

```
doctor - nurse : 0.6320
cat - dog : 0.7609
car - bus : 0.4693
king - queen : 0.6511
man - woman : 0.7664
apple - orange : 0.3920
teacher - student : 0.6301
paris - france : 0.5551
india - china : 0.3533
computer - laptop : 0.6640
```

```
print("""
Find top 5 similar words for selected words
```

```

"""
# Define a list of words for which to find similar terms
selected_words = ["king", "university", "doctor", "india", "computer"]

for word in selected_words:
    print(f"\nTop 5 words similar to '{word}':")
    similar_words = model.most_similar(word, topn=5) # Find the top 5 most similar words
    for w, score in similar_words:
        print(f"{w} : {score:.4f}") # Print each similar word and its similarity score

```

Find top 5 similar words for selected words

Top 5 words similar to 'king':

```

kings : 0.7138
queen : 0.6511
monarch : 0.6413
crown_prince : 0.6204
prince : 0.6160

```

Top 5 words similar to 'university':

```

universities : 0.7004
faculty : 0.6781
university : 0.6758
undergraduate : 0.6587
univeristy : 0.6585

```

Top 5 words similar to 'doctor':

```

physician : 0.7806
doctors : 0.7477
gynecologist : 0.6948
surgeon : 0.6793
dentist : 0.6785

```

Top 5 words similar to 'india':

```

indian : 0.6967
usa : 0.6836
pakistan : 0.6815
chennai : 0.6676
america : 0.6589

```

Top 5 words similar to 'computer':

```

computers : 0.7979
laptop : 0.6640
laptop_computer : 0.6549
Computer : 0.6473
com_puter : 0.6082

```

```

print("""
Perform analogy tasks using vector arithmetic
""")

```

Define a list of analogy tasks (positive words, negative words)

```

analogies = [
    (["king", "woman"], ["man"]),
    (["paris", "india"], ["france"]),
    (["teacher", "hospital"], ["school"])
]

```

for positive, negative in analogies:

```

# Perform the analogy calculation (e.g., king - man + woman = queen)
result = model.most_similar(positive=positive, negative=negative, topn=1)
print(f"\n{positive} - {negative} = {result}") # Print the analogy and its closest result

```

Perform analogy tasks using vector arithmetic

```
['king', 'woman'] - ['man'] = [('queen', 0.7118193507194519)]
```

```
['paris', 'india'] - ['france'] = [('chennai', 0.5442505478858948)]
```

```
['teacher', 'hospital'] - ['school'] = [('Hospital', 0.6331106424331665)]
```

```

print("""
Visualize selected word embeddings using PCA
""")

```

Define a list of words to visualize

```

words = ["king", "queen", "man", "woman", "doctor", "nurse",
         "paris", "france", "india", "delhi",
         "apple", "orange", "cat", "dog"]

```

Get the Word2Vec vectors for the selected words

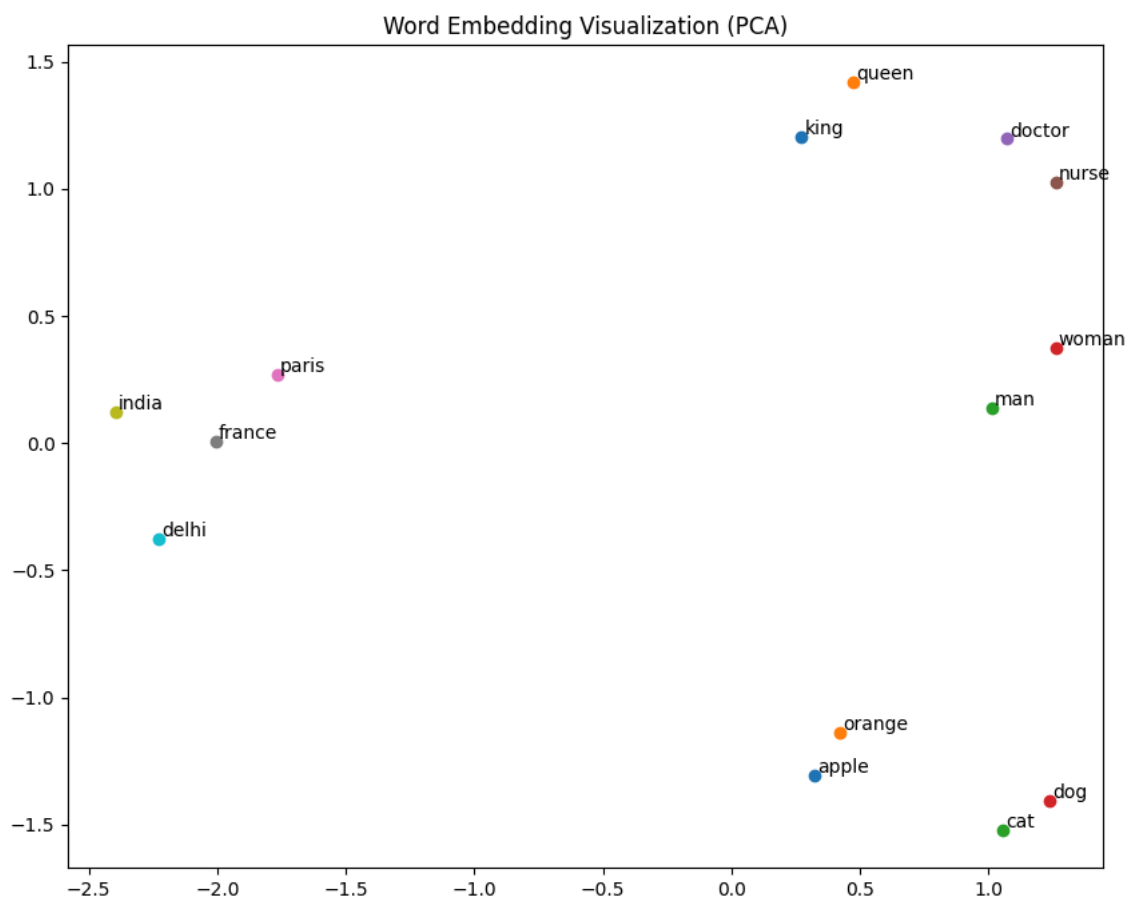
```
word_vectors = [model[word] for word in words]

# Initialize PCA to reduce dimensionality to 2 components
pca = PCA(n_components=2)
# Fit PCA to the word vectors and transform them to 2D
reduced = pca.fit_transform(word_vectors)

# Create a new figure for plotting
plt.figure(figsize=(10,8))
# Iterate through reduced vectors and words to plot them
for i, word in enumerate(words):
    plt.scatter(reduced[i,0], reduced[i,1]) # Plot the 2D point
    plt.text(reduced[i,0]+0.01, reduced[i,1]+0.01, word) # Add the word as text near the point

# Set the title of the plot
plt.title("Word Embedding Visualization (PCA)")
# Display the plot
plt.show()
```

Visualize selected word embeddings using PCA



Start coding or [generate](#) with AI.

